

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчёт

Лабораторная работа 2

“Реализация микросервисной архитектуры”

Выполнил:

Шестак Б.Е.

К3340

Проверил:

Добряков Д.И.

Санкт-Петербург

2026 г.

Задача

На основе документа, сформированного в рамках ДЗ4, реализовать разделение монолитного бэкенд-приложения платформы поиска работы на микросервисы с соблюдением концепции database-per-service, настроить межсервисное взаимодействие и API Gateway.

1. Структура микросервисов

На основе архитектурного проекта из ДЗ4 монолит разделён на четыре независимых микросервиса и API Gateway. Каждый сервис имеет собственную базу данных, набор моделей и независимый запуск.

Микросервис	Порт	База данных	Ответственность
auth-service	8001	auth.db	Регистрация, аутентификация, профили пользователей
vacancy-service	8002	vacancy.db	Каталог вакансий, компании, навыки
resume-service	8003	resume.db	Резюме соискателей, опыт работы, образование
application-service	8004	application.db	Отклики на вакансии, история, статусы
api-gateway	8000	—	Единая точка входа, прокси запросов к сервисам

2. Разделение базы данных

Каждый сервис использует собственную базу данных SQLite. Внешние ключи между базами заменены на идентификаторы без ограничений целостности. Ссылочная целостность проверяется на уровне приложения через внутренние HTTP-запросы.

auth.db содержит таблицу users (8 полей: первичный ключ, email, хэш пароля, роль, имя, фамилия, телефон, дата создания).

vacancy.db содержит четыре таблицы: skills (справочник), companies (компании с owner_id), vacancies (центральная) и vacancy_skills (ассоциативная M:N). Все зависимые таблицы связаны с vacancies через внешний ключ внутри сервиса.

resume.db содержит четыре таблицы: resumes (центральная), work_experience и education (опыт и образование, связаны через resume_id), resume_skills (навыки резюме с уровнем). Поле user_id ссылается на auth-service без DB FK.

application.db содержит одну таблицу applications с полями vacancy_id, user_id, resume_id — числовые ссылки на сущности из других сервисов.

3. Структура каждого сервиса

Все четыре сервиса построены по единому шаблону на Python + FastAPI + SQLAlchemy:

```
service/
├─ requirements.txt    # зависимости
├─ Dockerfile          # контейнеризация
└─ app/
```

```

└─ main.py          # FastAPI app, create_all, подключение
роутера
└─ database.py      # подключение к своей БД (SQLite)
└─ models.py        # SQLAlchemy ORM (только таблицы сервиса)
└─ schemas.py       # Pydantic схемы запросов и ответов
└─ dependencies.py  # межсервисные вызовы (httpx)
└─ routers.py       # все эндпоинты сервиса

```

auth-service содержит одну модель (User) и логику JWT (python-jose + bcrypt через passlib).

vacancy-service содержит четыре модели (Skill, Company, Vacancy, VacancySkill) и эндпоинт GET /internal/vacancies/{id} для межсервисного взаимодействия.

resume-service содержит четыре модели (Resume, WorkExperience, Education, ResumeSkill). Предоставляет GET /resumes/internal/{id}.

application-service содержит одну модель (Application) без декораторов связей к другим сервисам.

4. Внешние эндпоинты

Каждый сервис предоставляет свою часть публичного API, соответствующую спецификации из ДЗ4:

auth-service (8001):

- POST /auth/register — регистрация с хэшированием пароля (bcrypt)
- POST /auth/login — вход с выдачей пары JWT-токенов
- POST /auth/refresh — обновление токенов по refresh_token
- GET /users/me — профиль текущего пользователя (защищён)
- PATCH /users/me — обновление профиля
- PUT /users/me/password — смена пароля

vacancy-service (8002):

- GET /skills — список навыков с поиском по имени
- GET /companies/{id}, POST /companies — управление компаниями
- GET /vacancies — список вакансий с фильтрацией и пагинацией
- POST /vacancies, GET/PUT/DELETE /vacancies/{id} — CRUD вакансий
- GET /vacancies/my — вакансии текущего работодателя

resume-service (8003):

- GET /resumes, POST /resumes — список и создание резюме
- GET/PUT/DELETE /resumes/{id} — CRUD резюме
- POST /resumes/{id}/experience — добавление опыта работы
- POST /resumes/{id}/education — добавление образования
- PUT /resumes/{id}/skills — обновление навыков резюме

application-service (8004):

- POST /vacancies/{id}/apply — создание отклика с межсервисными проверками
- GET /applications/my — история откликов соискателя (защищён)
- GET /vacancies/{id}/applications — отклики на вакансию (только для работодателя)
- PATCH /applications/{id}/status — изменение статуса отклика

5. Внутренние эндпоинты (межсервисное взаимодействие)

Для обеспечения проверок существования сущностей между сервисами реализованы внутренние эндпоинты, недоступные через API Gateway:

auth-service:

- GET /internal/validate?token=... — возвращает { user_id, role, email } или 401

vacancy-service:

- GET /internal/vacancies/{vacancy_id} — проверка вакансии с данными о компании и активности

resume-service:

- GET /resumes/internal/{resume_id} — проверка резюме, включая user_id для верификации владельца

6. Межсервисное взаимодействие

При создании отклика application-service выполняет три внутренних HTTP-запроса:

1. К auth-service — проверка JWT токена (GET /internal/validate)
2. К vacancy-service — проверка существования и активности вакансии (GET /internal/vacancies/{id})
3. К resume-service — проверка резюме и его принадлежности текущему пользователю (GET /resumes/internal/{id})

При работе с вакансиями vacancy-service проверяет токен у auth-service. Аналогично поступает resume-service при работе с резюме.

Все внутренние запросы выполняются асинхронно через `httpx.AsyncClient`. При недоступности сервиса возвращается ошибка 503 Service Unavailable.

7. API Gateway

API Gateway реализован на FastAPI (порт 8000) с использованием `httpx` в качестве `reverse proxy`. Все запросы маршрутизируются по префиксу URL:

- `/auth/*` и `/users/*` → `auth-service` (8001)
- `/skills/*`, `/companies/*`, `/vacancies/*` → `vacancy-service` (8002) (кроме `/apply` и `/applications`)
- `/resumes/*` → `resume-service` (8003)
- `/applications/*` и `/vacancies/{id}/apply` → `application-service` (8004)

Gateway пробрасывает HTTP-метод, заголовки (включая `Authorization`) и тело запроса. При недоступности сервиса возвращает 503 с описанием ошибки.

8. Запуск

Все сервисы оркестрированы через `docker-compose.yml`. Порядок запуска управляется через `depends_on` и `healthcheck`:

```
docker-compose up --build

# Swagger UI доступен по адресам:
http://localhost:8000/docs  - API Gateway
http://localhost:8001/docs  - auth-service
http://localhost:8002/docs  - vacancy-service
http://localhost:8003/docs  - resume-service
http://localhost:8004/docs  - application-service
```


9. Тестирование

Проведено сквозное тестирование через API Gateway (порт 8000):

- Регистрация нового пользователя — успешно (201)
- Вход в систему — успешно (200), получен JWT-токен
- Создание вакансии работодателем — успешно (201), токен проверен у auth-service
- Создание резюме соискателем — успешно (201)
- Создание отклика — успешно (201), межсервисная проверка вакансии и резюме отработала
- Просмотр истории откликов — успешно (200)
- Отклик на несуществующую вакансию — ошибка 404 (application-service проверил через vacancy-service)
- Отклик с чужим резюме — ошибка 403 (resume-service вернул user_id, несовпадение обнаружено)
- Межсервисное взаимодействие работает корректно

Вывод

В результате выполнения лабораторной работы монолитное бэкенд-приложение успешно разделено на четыре независимых микросервиса: auth-service, vacancy-service, resume-service и application-service. Каждый сервис использует собственную базу данных (database-per-service). Реализовано синхронное HTTP-взаимодействие между сервисами через внутренние эндпоинты для проверки целостности данных. API Gateway обеспечивает единую точку входа для клиентов. Проведено тестирование, подтверждающее корректную работу всех сервисов и межсервисных проверок.